

In [1]:

```
#numpy  
#NumPy ufuncs (universal functions)
```

In [4]:

```
list1 = [10,20,30,40,50]  
list2 = [60,70,80,90,100]
```

```
list3 = []
```

```
for var in list1:  
    list3.append(var)  
for var in list2:  
    list3.append(var)  
print(list3)
```

```
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

In [5]:

```
#zip() method  
list4 = []  
for var1, var2 in zip(list1, list2):  
    list4.append(var1 + var2)  
print(list4)
```

```
[70, 90, 110, 130, 150]
```

In [6]:

```
#ufunc numpy -- > add()  
import numpy as np  
list5 = np.add(list1,list2)  
print(list5)
```

```
[ 70  90 110 130 150]
```

In [15]:

```
list6 = []  
  
for i in range(0,len(list1)):  
    list6.append(list1[i] + list2[i])  
print(list6)
```

```
[70, 90, 110, 130, 150]
```

In [21]:

*#add user defined func as an Numpy ufunc library***import** numpy as np

```
def myaddition(list1, list2):
    return list1+list2
```

```
myaddition = np.frompyfunc(myaddition, 2, 1)  #frompyfunc(myfunction, input, output)
```

```
print(myaddition([1, 2, 3, 4], [5, 6, 7, 8]))
print(type(myaddition))
```

```
[6 8 10 12]
<class 'numpy.ufunc'>
```

In [29]:

*#ufunc arithmetic function***import** numpy as np

```
var1 = np.array([2,35,6,78,8])
var2 = np.array([45,787,43,456,8])
```

```
print("add()")
varadd = np.add(var1, var2)
print(varadd)
print(type(varadd))
```

```
print("subtract()")
varsub = np.subtract(var1, var2)
print(varsub)
print(type(varsub))
```

```
print("multiply()")
varmul = np.multiply(var1, var2)
print(varmul)
print(type(varmul))
```

```
print("divide()")
vardiv = np.divide(var2, var1)
print(vardiv)
print(type(vardiv))
```

```
add()
[ 47 822  49 534  16]
<class 'numpy.ndarray'>
subtract()
[ -43 -752  -37 -378    0]
<class 'numpy.ndarray'>
multiply()
[   90 27545   258 35568   64]
<class 'numpy.ndarray'>
divide()
[22.5         22.48571429  7.16666667  5.84615385  1.         ]
<class 'numpy.ndarray'>
```

In [38]:

```
#power
num1 = np.array([2,3,4,5,6,7])
num2 = np.array([2,3,2,2,2,3])

print('power')
numpow = np.power(num1, num2)
print(numpow)

print("Remainder")
numrem = np.mod(num1, num2)
print(numrem)

print("Remainder")
numrem1 = np.remainder(num1, num2)
print(numrem1)
```

```
power
[ 4 27 16 25 36 343]
Remainder
[0 0 0 1 0 1]
Remainder
[0 0 0 1 0 1]
```

In [39]:

```
#Quotient and Mod
quomod = np.divmod(num1,num2)
print(quomod)
```

```
(array([1, 1, 2, 2, 3, 2], dtype=int32), array([0, 0, 0, 1, 0, 1], dtype=i
nt32))
```

In [40]:

```
#absolute
num3 = [45,67,-32,56,-5,-7]
absolute = np.absolute(num3)
print(absolute)
```

```
[45 67 32 56  5  7]
```

In [41]:

```
#decimal numbers
declist = [0.4536, 23.54, 56.889,5.6788]
print(declist)
```

```
[0.4536, 23.54, 56.889, 5.6788]
```

In [43]:

```

arrtrun = np.trunc(declist)
print(type(arrtrun))
print(arrtrun)

arrfix = np.fix(declist)
print(type(arrfix))
print(arrfix)

```

```

<class 'numpy.ndarray'>
[ 0. 23. 56.  5.]
<class 'numpy.ndarray'>
[ 0. 23. 56.  5.]

```

In [44]:

```

var = np.around(45.56673, 3)
print(var)

```

45.567

In [49]:

```

print(declist)

print('Floor')
print(np.floor(declist))
print('Ceil')
print(np.ceil(declist))

```

```

[0.4536, 23.54, 56.889, 5.6788]
Floor
[ 0. 23. 56.  5.]
Ceil
[ 1. 24. 57.  6.]

```

In [59]:

```

#product

import numpy as np
arr1 = np.array([5,4,3,2,1])
result = np.prod(arr1)
print(result)

arr2 = np.array([1,2,3,4,5])
res = np.prod([arr1,arr2])
print(res)

```

```

120
14400

```

In [60]:

```

ressep = np.prod([arr1,arr2], axis = 1)
print(ressep)

```

[120 120]

In [63]:

```
#Cumulative Product
print(arr1)
cumprod = np.cumprod(arr1)
print(cumprod)
```

```
[5 4 3 2 1]
[ 5 20 60 120 120]
```

In [70]:

```
#NumPy Differences

num = [34,64,24,7,0,544,67,89]
diffsucc = np.diff(num)
print(diffsucc)

diffsuccn = np.diff(num, n=2)
print(diffsuccn)
```

```
[ 30 -40 -17  -7 544 -477  22]
[ -70  23  10  551 -1021  499]
```

In [71]:

```
num = np.array([10, 15, 25, 5])
diffsucc = np.diff(num)
print(diffsucc)

diffsuccn = np.diff(num, n=2)
print(diffsuccn)
```

```
[ 5 10 -20]
[ 5 -30]
```

In [73]:

```
#LCM

var1 = 15
var2 = 5

lcm = np.lcm(var1,var2)
print(lcm)
```

15

In [74]:

```
import numpy as np
var = np.array([4,8,12,16])
lcmvar = np.lcm.reduce(var)
print(lcmvar)
```

48

In [75]:

```
#GCD / HCF
import numpy as np

num1 = 6
num2 = 9

x = np.gcd(num1, num2)

print(x)
```

3

In [76]:

```
import numpy as np

arr = np.array([20, 8, 32, 36, 16])

x = np.gcd.reduce(arr)

print(x)
```

4

In []: